

Forge Flow

Differential certificates, optimal cofactor ladders, and exact analytic witnesses

A substantive analytic extension to the Forge Lean tactic design

Central result. In a precisely specified exponential-polynomial certificate language, feasibility does not require a permutation search, and a shortest certificate can be constructed by a binary search over one integer. The resulting certificates contain rational arithmetic, exact derivative identities, and nonnegative initial values—not floating-point evidence.

Additional workers. Coupled differential-inequality certificates, exact rational exponential enclosures, point counterexamples, and existence-and-uniqueness certificates for roots in an externally specified interval.

Delivered evidence. Executed Python implementations, separate dense and sparse algebra paths, replay without site packages, finite exhaustive and symbolic cross-checks, corruption tests, and recorded certificates. No Lean compilation or tactic benchmark was performed in this environment.

Abstract

Forge already proposes structural proof planning, arithmetic witnesses, finite closure certificates, nonlinear polynomial reasoning, and several external-solver interfaces. This article does not propose those mechanisms again. It develops an analytic lane for statements involving real exponentials and hyperbolic functions, with three concrete certificate families.

The first family is an anchored cofactor ladder: repeated application of $D - c$ transforms an exponential polynomial into an ordinary polynomial with nonnegative coefficients, while every intermediate value at zero is nonnegative. A scalar integrating-factor theorem proves soundness. An adjacent-exchange argument proves that increasing cofactor order is sufficient for feasibility within an input-spectrum grammar. A multiplicity argument then reduces shortest-certificate construction to the number of zero cofactors, a monotone one-dimensional feasibility problem. The prototype implements this deterministic compiler and a checkable minimality receipt.

The second family certifies coupled residuals through a positive differential system with a rational Metzler matrix. The third supplies exact rational enclosures, point counterexamples, and root brackets whose derivative-sign covers prove uniqueness in the stated interval. These workers have deliberately different logical outputs.

The recorded corpus contains 196 positive ladder cases, 40 coupled-system cases, 44 negative-point cases, 8 true functions rejected by the ladder grammar, and 16 isolated-root cases. The optimal compiler reduces the total number of cofactor steps from 1,212 to 288 on the same 196 positive inputs. All 624 targeted invalid mutations are rejected. Finite independent checks cover 200 random small permutation problems and 200 symbolic derivatives. These are research-code results, not Lean-kernel proofs and not evidence of superiority over any existing tactic. The article gives the mathematical proofs, algorithms, concrete library targets, source-to-goal obligations, limitations, and acceptance gates needed to turn the lane into a genuine Forge extension.

Contents

1	The capability delta, not another architecture	1
1.1	What is already in Forge	1
1.2	Three additions and their precise boundaries	1
1.3	What “new” means in this article	1
1.4	Evidence levels	2
2	The exact analytic fragment	2
2.1	A representation closed under all required operations	2
2.2	Why the origin and right ray are part of the problem	3
2.3	Input expansion and its resource boundary	3
3	Anchored cofactor ladders	4
3.1	The one-step rule	4
3.2	What the checker actually recomputes	4
3.3	Strict positivity and zeros	5
3.4	High-order contact is not a numerical singularity	5
4	From a search problem to an optimal compiler	6
4.1	The finite grammar to which completeness applies	6

4.2	Exchange makes increasing order sufficient	6
4.3	Nonzero modes force their complete multiplicities	7
4.4	A checkable minimality receipt	9
4.5	Complexity and the correct sense of optimality	9
5	Worked certificates and what they buy	9
5.1	Arbitrarily high Taylor truncations need one step	9
5.2	An exponential majorant for binomial powers	10
5.3	A fully expanded four-step certificate	10
5.4	Hyperbolic inequalities with exact contact at zero	11
5.5	A rational envelope without floating point	11
5.6	An explicit success language, not a hidden universal claim	12
6	Coupled differential certificates	12
6.1	Why a vector rule is useful	12
6.2	The implemented certificate and row algorithm	13
6.3	A parametric family behind the 40 tests	14
6.4	A scalable replacement for coefficient enumeration	14
7	Exact analytic enclosures and point counterexamples	15
7.1	A small exponential enclosure with a proved tail	15
7.2	Interval evaluation of the original normal form	16
7.3	A point certificate has a different logical meaning	16
8	Root certificates as analytic witness plans	16
8.1	The target includes the interval	16
8.2	Endpoint signs plus an exact derivative cover	17
8.3	Concrete tested root problems	17
8.4	An exact existence proof is not a computable real value	17
8.5	Deliberate incomplete cases	18
9	A concrete Lean implementation plan	18
9.1	Keep discovery outside the trusted mathematics	18
9.2	Suggested module boundaries	18
9.3	The first four formal theorems	19
9.4	Concrete Mathlib targets inspected	20
9.5	The source bridge is part of the proof	20
9.6	Routing and result types specific to this lane	21
9.7	How Leant and Djex would use the new worker	21
9.8	Trust and performance gates	22

10 Executed evidence and its limits	22
10.1 Environment and record locations	22
10.2 Results by logical output	22
10.3 The cofactor-order and compression ablation	23
10.4 Targeted invalid certificates	23
10.5 Independent finite checks	24
10.6 Replay without discovery or site packages	24
10.7 What remains unmeasured	25
11 Implementation sequence and acceptance gates	25
12 Reproduction and package guide	26
12.1 Files	26
12.2 Commands	26
12.3 Small direct API example	27
13 Conclusion	27
References	28

1 The capability delta, not another architecture

1.1 What is already in Forge

The baseline is Forge commit `c98e47c5f804e92880fc1d0e378c1b95832b685c`. Its README describes a merged design from eighteen proposals, not an installed tactic. Its status ledger includes exact polynomial-cone certificates, Bernstein subdivision, univariate Sturm-based reasoning, structural induction, arithmetic witness synthesis, observable-space and ideal closures, finite-algebra covers, Kripke countermodels, boundary-safe telescoping, and Ore recurrence transport. Its merge also records elaboration of sixteen core-only Lean files; this does not mean that its complete proposed tactic or its Mathlib-dependent certificate checkers have been implemented [1, 2, 4].

Those contributions are prerequisites here. In particular, merely suggesting “add invariants,” “use a certificate checker,” “synthesize an auxiliary lemma,” or “consult Leant and Djex” would duplicate the current repository. The baseline nonlinear chapter targets polynomial expressions over $\mathbb{R}$; nonpolynomial real subexpressions can be treated as opaque variables, but additional relations about them must arrive as proved constraints [3]. This is the opening addressed by *Forge Flow*.

For example, treating $y = e^x$ as an unconstrained real variable cannot establish

$$e^{2x} - (2 + x^2)e^x + 1 \geq 0 \quad (x \geq 0).$$

A polynomial worker may eventually solve the goal after suitable analytic lemmas are supplied. Flow’s job is to produce those lemmas from exact differential structure, rather than hope that the appropriate inequality is already in the context.

1.2 Three additions and their precise boundaries

Addition		Concrete mechanism	Not a claim made here
Cofactor compiler		Rational exponential-polynomial normalization; exact $D - c$ ladders; increasing-order feasibility; binary-search minimum	A decision procedure for all nonnegative exponential polynomials
Coupled worker	residual	Enumerate rational matrix rows; check a Metzler differential system and coefficient-positive forcing	Unrestricted invariant synthesis or general ODE solving
Analytic worker	witness	Exact Taylor-tail enclosures; rational point refutations; interval endpoint signs plus derivative coverage	A rational value of a transcendental root, or global uniqueness from a local bracket

Table 1: The new lane has three distinct outputs, with different mathematical meanings.

The most substantial algorithmic result is the shortest-ladder compiler in Section 4. It replaces an initially implemented breadth-first search over a multiplicity grid by a deterministic feasibility pass and a binary search. The grid search remains in the package as a small-instance oracle and an ablation, not as the proposed production algorithm.

1.3 What “new” means in this article

The novelty claim is a *delta against the reviewed Forge materials*: the README, implementation-status ledger, nonlinear chapter, provenance map, and the neighboring-project transfer document

at the pinned revision. This was not a line-by-line audit of every archived submission. A GitHub code-search response was explicitly marked incomplete and was not treated as proof of absence. The package records this review boundary in `docs/PROVENANCE.md`.

Nor is analytic proof automation itself new. Rocq’s Interval package already supports elementary functions, automatic differentiation, Taylor models, and specialized root reasoning. MetiTarski combines resolution with real-algebraic reasoning to prove inequalities involving elementary functions [13, 14]. Differential invariants and differential ghosts provide a much broader logical context for reasoning from differential equations [12]. Positivity and derivative constructions for exponential-polynomial spaces also belong to the established extended-Chebyshev literature [15].

The contribution here is a particular small certificate language, a complete and optimal compiler *for that language*, executable checking paths, and a concrete Lean integration plan. The exchange and minimality arguments are derived in full rather than asserted to be a previously unknown theorem. No worldwide priority claim is needed for them to be useful to Forge.

1.4 Evidence levels

Component	Status in this delivery
Dense arithmetic, sparse replay, optimal ladders	Implemented and executed in Python
Constant-matrix residual synthesis	Implemented and executed for supplied residual vectors
Rational enclosures and root certificates	Implemented and executed in the stated bounded numerical fragment
Soundness, exchange, minimality, and positive-system results	Mathematical proofs in this article; not formalized here
Lean normalizer, reflected checker, and source bridge	Specified, not implemented
Lean acceptance targets	Supplied as proposition definitions, not proofs; uncompiled
Comparison with <code>grind</code> , <code>Aesop</code> , or other tactics	Not run

Table 2: Python success is not silently promoted to Lean acceptance.

2 The exact analytic fragment

2.1 A representation closed under all required operations

Let $\mathcal{E}$ be the ring of real functions represented by

$$f(x) = \sum_{\lambda \in \Lambda} P_\lambda(x) e^{\lambda x}, \quad \Lambda \subset \mathbb{Q} \text{ finite}, \quad P_\lambda \in \mathbb{Q}[x]. \quad (1)$$

Zero coefficient polynomials are removed and the remaining rates are stored in strictly increasing order. Within each polynomial, coefficients are ascending by degree. The empty sum is the zero function. Products are normalized by adding rates, multiplying polynomial coefficients, and collecting equal rates. Thus products of $\sinh(rx)$ and $\cosh(rx)$ reduce to this representation using their exponential definitions.

For a rational cofactor c , define

$$T_c f = (D - c)f = \sum_{\lambda \in \Lambda} (P'_\lambda + (\lambda - c)P_\lambda) e^{\lambda x}. \quad (2)$$

No transcendental arithmetic is required to compute this representation. The anchor is also rational:

$$a(f) := f(0) = \sum_{\lambda \in \Lambda} P_\lambda(0). \quad (3)$$

Every denotation is smooth on all of $\mathbb{R}$. Exact rational arithmetic is used for normalization, derivatives, anchor signs, equality tests, and terminal tests.

The terminal cone is intentionally elementary:

$$\mathcal{C}_+ = \left\{ \sum_{j=0}^d b_j x^j : b_j \in \mathbb{Q}_{\geq 0} \right\}.$$

Every member is nonnegative on $[0, \infty)$. Testing membership is a coefficient check. This is *not* the full cone of nonnegative polynomials: $(x-1)^2$ is nonnegative but not in $\mathcal{C}_+$ in the monomial basis. Keeping this terminal test fixed is essential to the exact shortest-certificate theorem.

2.2 Why the origin and right ray are part of the problem

The implemented positivity problem is

$$\forall x \in \mathbb{R}, \quad 0 \leq x \implies 0 \leq f(x). \quad (4)$$

Its serialization includes the anchor 0 and the direction **right**. A checker rejects an altered anchor or direction; it does not accept a certificate for an easier domain selected by the producer.

The origin is not a cosmetic choice. At a general rational anchor a , the values $e^{\lambda a}$ need not be rational, so a coefficient-only initial-value check would be wrong. A left-ray theorem can be reduced to the implemented problem by the exact substitution $x = -t$, accompanied by a Lean proof of the substitution. A general translation needs either symbolic initial-value proofs or analytic enclosures, and has not been implemented as a positivity front end here.

There is nevertheless a useful large application surface without generalizing the normal form. Once Lean has established $\forall t \geq 0, f(t) \geq 0$, it may specialize the theorem at any expression $t = u$ for which $0 \leq u$ is proved. For instance, $u = (\log y)^2$ does not require a logarithm differentiation engine: the square supplies the domain proof, and the theorem is instantiated. This is ordinary theorem application with an opaque argument, not a claim that logarithms have been added to the reifier.

2.3 Input expansion and its resource boundary

A source normalizer should recognize rational constants, one selected real expression, addition, multiplication, nonnegative integer powers, and exponentials with rational multiples of that selected expression as arguments. Hyperbolic functions are a proved preprocessing layer. It must refuse an unsupported expression rather than assign it an unintended meaning.

Expansion can be expensive even though each later ladder step is cheap. If the input is a product of sums of exponentials, distinct rate sums can produce many modes. The front end therefore needs separate limits on source-expression size, number of modes, polynomial degree, total coefficient count, and coefficient bit length. These are not interchangeable with the number of cofactor steps. The Python checker currently limits rate entries to 512, degree to 256, and ladder length to 512; these are engineering caps, not mathematical bounds on the theory.

3 Anchored cofactor ladders

3.1 The one-step rule

A common obstacle in inequality proofs is that g' is not nonnegative even when g is. The more flexible condition is $g' - cg \geq 0$: multiplying g by a positive exponential removes the unwanted multiple of g .

Lemma 3.1 (Scalar cofactor rule). *Let $g : [0, \infty) \rightarrow \mathbb{R}$ be continuously differentiable and $c \in \mathbb{R}$. If*

$$g(0) \geq 0, \quad g'(x) - cg(x) \geq 0 \quad (x \geq 0),$$

then $g(x) \geq 0$ for every $x \geq 0$.

Proof. For every $x \geq 0$,

$$\frac{d}{dx}(e^{-cx}g(x)) = e^{-cx}(g'(x) - cg(x)) \geq 0.$$

On any compact interval $[0, X]$, the mean value theorem implies that $e^{-cx}g(x)$ is nondecreasing. Its value at zero is nonnegative. Since $e^{-cx} > 0$, so is $g(x)$. The argument applies to every finite X . $\square$

This is a theorem about the whole half-line, not a numerical test of a sampled trajectory. It also imposes no sign condition on c . In particular, negative cofactors are mathematically legitimate and important for expressions containing e^{-x} .

Definition 3.2 (Ladder certificate). A ladder for $f \in \mathcal{E}$ consists of rational cofactors $c_0, \dots, c_{k-1}$ and the implicitly defined chain

$$g_0 = f, \quad g_{i+1} = T_{c_i}g_i.$$

It is accepted when $g_i(0) \geq 0$ for $0 \leq i \leq k$ and $g_k \in \mathcal{C}_+$.

Theorem 3.3 (Ladder soundness). *Every accepted ladder establishes (4).*

Proof. The terminal polynomial is nonnegative on the right ray. If g_{i+1} is nonnegative, the identity $g'_i - c_i g_i = g_{i+1}$ and the checked anchor permit Lemma 3.1. Apply the lemma backwards for $i = k-1, \dots, 0$. $\square$

3.2 What the checker actually recomputes

The producer need not supply intermediate functions. A compact certificate stores the cofactors, all rational anchors, and the terminal normal form. The checker starts from the *externally provided problem*, recomputes each T_c transition, and verifies that each stored anchor is exactly the anchor it recomputed. It finally requires exact equality with the claimed terminal normal form and coefficient nonnegativity.

```

checkLadder(problem, certificate):
  g := decodeOriginalExpPolynomial(problem)
  require problem.anchor = 0 and problem.direction = right
  require numberOfAnchors = numberOfCofactors + 1
  for i = 0 .. numberOfCofactors:
    require certificate.anchor[i] = exactValueAtZero(g)
    require certificate.anchor[i] >= 0
    if i < numberOfCofactors:
      g := exactDerivative(g) - certificate.cofactor[i] * g
  require g = decode(certificate.terminal)
  require every remaining rate is zero
  require every polynomial coefficient is nonnegative
  accept

```

The rational anchors are redundant from a mathematical standpoint, but useful as auditable receipts and corruption targets. They cannot replace the recomputation. A certificate with all anchors edited to zero is rejected whenever those are not the actual values.

The Python discovery implementation stores a dense coefficient array for each exponential mode. The ladder checker stores a separate sparse map keyed by (λ, j) and implements its own derivative and linear-combination arithmetic. It imports neither the discovery module nor the dense model. This separation catches more than replaying the producer's own transition routine, although both implementations still depend on Python and its rational-number library. Neither has a formal correctness proof.

3.3 Strict positivity and zeros

Proposition 3.4 (A useful semantic limitation). *If a ladder is accepted, then either its denoted function is identically zero or it is strictly positive at every $x > 0$.*

Proof. A nonzero member of $\mathcal{C}_+$ is strictly positive for $x > 0$. For a step $g' - cg = h$, variation of constants gives

$$g(x) = e^{cx} \left(g(0) + \int_0^x e^{-ct} h(t) dt \right). \quad (5)$$

If h is strictly positive on $(0, \infty)$, the integral is strictly positive for every $x > 0$. If h is identically zero, then $g(x) = g(0)e^{cx}$, which is either identically zero or strictly positive. Starting at the terminal polynomial and applying this dichotomy backwards proves the statement. $\square$

This proposition explains a real limitation, not a bug. A nonzero function with an interior zero, such as $(e^x - 2)^2$, cannot have a certificate in this ladder language, even if arbitrary rational cofactors were allowed. An existing square or polynomial-cone worker can prove its nonnegativity immediately after setting $y = e^x$. Flow should send such a problem to that worker, not relax the anchor checks.

The prototype's accepted output is a nonnegativity certificate. The proposition supplies a mathematically justified strictness extension, but a Lean strictness-producing adapter would still need its own formal theorem and a checked nonzero condition. The current output does not silently advertise that stronger interface.

3.4 High-order contact is not a numerical singularity

Suppose $f(0) = f'(0) = \dots = f^{(m-1)}(0) = 0$. Natural interval evaluation of a difference of nearly equal exponentials may be inconclusive near zero, and a proof based on strict positivity

margins can struggle with the boundary. A ladder records these zeros exactly. They cost rational equalities, not additional numerical precision. The hyperbolic examples below have sixth- and seventh-order contact and remain short exact certificates.

This is one reason to keep the symbolic ladder worker and the numerical enclosure worker separate. The former is especially attractive at exact contacts and on unbounded rays. The latter is useful for strict inequalities on bounded intervals, concrete counterexamples, and root brackets. Neither should be forced to emulate the other's strengths.

4 From a search problem to an optimal compiler

4.1 The finite grammar to which completeness applies

For each nonzero coefficient polynomial in (1), put

$$m_\lambda = 1 + \deg P_\lambda, \quad N = \sum_{\lambda \in \Lambda} m_\lambda.$$

Define $m_0 = 0$ when the rate zero is absent. The full operator

$$L_f = \prod_{\lambda \in \Lambda} (D - \lambda)^{m_\lambda} \quad (6)$$

annihilates f . This follows immediately from

$$(D - \lambda)^j (P_\lambda e^{\lambda x}) = P_\lambda^{(j)} e^{\lambda x}$$

and commutation of constant-coefficient differential operators.

Definition 4.1 (Input-spectrum polynomial-terminal grammar). A ladder belongs to $\mathcal{G}(f)$ if each cofactor belongs to the input rate set Λ , rate λ occurs at most m_λ times, and its terminal test is exactly membership in $\mathcal{C}_+$. Early stopping is allowed. The zero function has the empty ladder.

The sound checker can accept rational cofactors outside this grammar. The following completeness and optimality theorems cannot. In particular, they do not allow extra cofactor rates, more than the input multiplicities, a different anchor, or a stronger polynomial terminal prover.

The initial implementation explored states $(k_\lambda)_{\lambda \in \Lambda}$ with $0 \leq k_\lambda \leq m_\lambda$. Commutation makes the transformed expression depend only on this vector, so tabling avoids redundant permutations. Nevertheless, the grid has

$$\prod_{\lambda \in \Lambda} (m_\lambda + 1) \quad (7)$$

states. The next two results eliminate the need to traverse that grid.

4.2 Exchange makes increasing order sufficient

Lemma 4.2 (Adjacent exchange). *Suppose a valid portion of a ladder applies c and then d to a function h , with $c > d$. Replacing those two steps by d and then c preserves every anchor inequality and the function after the two steps.*

Proof. Write $a = h(0) \geq 0$ and $b = (T_c h)(0) \geq 0$. The new intermediate anchor is

$$(T_d h)(0) = (T_c h)(0) + (c - d)h(0) = b + (c - d)a \geq 0.$$

The two-step result is unchanged because $T_c T_d = T_d T_c$. All preceding and subsequent functions are therefore unchanged, except for the single intermediate function whose anchor was just checked. $\square$

Corollary 4.3 (Sorted feasibility). *For a fixed multiset of cofactors, if any ordering yields an accepted ladder, the increasing ordering does too.*

Proof. Repeatedly exchange adjacent inversions. The terminal function is unchanged by every exchange. $\square$

Notice that the result is one-sided. A decreasing ordering can fail even though increasing order succeeds. It is not correct to conclude that commutation makes the *intermediate anchors* order-independent. Only the final operator is order-independent; the inequality in the exchange lemma is the missing argument.

Lemma 4.4 (Completion to the full annihilator). *Every accepted ladder in $\mathcal{G}(f)$ extends to an accepted ladder using the full multiset in (6).*

Proof. Let the current terminal be $q \in \mathcal{C}_+$. Append all unused negative cofactors first. For $c < 0$, $T_c q = q' + |c|q$ still belongs to $\mathcal{C}_+$. Append the unused zero cofactors next. If a zero mode existed initially, the current polynomial degree is no larger than its original degree minus the number of zero factors already used. The remaining m_0 factors, accounting for those already used, therefore annihilate it. If no zero mode existed, the ordinary-polynomial terminal is already zero: none of the permitted operators creates a zero-rate mode. Finally append the unused positive cofactors, which act on zero. Every new anchor is nonnegative. $\square$

Theorem 4.5 (A deterministic feasibility test). *A ladder in $\mathcal{G}(f)$ exists if and only if every intermediate anchor in the full increasing-order annihilator chain is nonnegative. Thus a negative anchor in that canonical chain is a certificate that $\mathcal{G}(f)$ contains no accepted ladder.*

Proof. For the forward direction, complete a valid ladder by Lemma 4.4, then sort it by Corollary 4.3. For the reverse direction, the full chain has zero terminal by (6); if all anchors are nonnegative, it is an accepted ladder. $\square$

A production pass may stop earlier when it reaches $\mathcal{C}_+$. This does not weaken the theorem: such a prefix itself is a valid certificate. A failure receipt contains the negative anchor's index and its exact rational value. The replay path reconstructs the canonical chain up to that index.

The negative result is about the language. It says neither that f is negative somewhere nor that Lean cannot prove $f \geq 0$. Section 10 includes true functions for which the negative grammar result is the expected and correct outcome.

4.3 Nonzero modes force their complete multiplicities

The remaining question is whether a shorter ladder can be obtained by stopping before using every factor. This is where the special terminal cone gives a much stronger result.

Lemma 4.6 (Forced nonzero factors). *Every ladder in $\mathcal{G}(f)$ with an ordinary-polynomial terminal uses exactly m_λ copies of each nonzero input rate λ .*

Proof. Consider its action on a single nonzero mode $P_\lambda e^{\lambda x}$. Applying T_c with $c \neq \lambda$ replaces P by $P' + (\lambda - c)P$. If P has degree d and leading coefficient $a \neq 0$, the new polynomial has the same degree and leading coefficient $(\lambda - c)a \neq 0$. Such a step cannot eliminate the mode.

The factors with $c = \lambda$ differentiate its polynomial. By commutation, k such factors leave a nonzero polynomial precisely when $k \leq \deg P_\lambda$. Therefore eliminating the mode requires at least $\deg P_\lambda + 1 = m_\lambda$ copies. The grammar permits no more than this number. $\square$

Here “ordinary-polynomial terminal” is the normal-form condition used by the checker: all nonzero rates must be absent. The argument does not depend on deciding equality of arbitrary analytic expressions by sampling.

Consequently the only remaining choice is an integer

$$z \in \{0, 1, \dots, m_0\},$$

the number of zero cofactors. For each z , form the sorted list consisting of all mandatory nonzero factors and z zeros. Let $\Phi(z)$ mean that its anchors pass and its terminal lies in $\mathcal{C}_+$.

Lemma 4.7 (Monotonicity in the zero count). *For $0 \leq z < m_0$, $\Phi(z)$ implies $\Phi(z + 1)$.*

Proof. Append one zero cofactor to the accepted chain at z . Its terminal $q \in \mathcal{C}_+$ becomes $q' \in \mathcal{C}_+$, whose anchor is nonnegative. Sort the enlarged cofactor list by the exchange lemma. The result is exactly the candidate tested by $\Phi(z + 1)$. $\square$

Theorem 4.8 (Shortest-ladder compiler). *After a successful canonical feasibility pass, binary search for the least z such that $\Phi(z)$ holds produces a shortest ladder in $\mathcal{G}(f)$. Its number of steps is*

$$\sum_{\lambda \neq 0} m_\lambda + z_{\min}. \quad (8)$$

Proof. Canonical success guarantees $\Phi(m_0)$ by Theorem 4.5. Lemma 4.7 makes Φ monotone. Any accepted ladder must use the mandatory nonzero factors by Lemma 4.6; sorting preserves its acceptance, so its zero count is a feasible value of Φ . Conversely, the candidate at $z_{\min}$ is an accepted ladder. Formula (8) is therefore both a lower bound and an attained length. $\square$

```

optimalLadder(f):
  result := increasingAnnihilatorPass(f)
  if result contains a negative canonical anchor:
    return NoCertificateInThisGrammar(result.obstruction)

  lo := 0
  hi := multiplicityOfZeroRate(f)
  while lo < hi:
    mid := floor((lo + hi) / 2)
    if sortedMandatoryFactorsAndZeros(f, mid) passes:
      hi := mid
    else:
      lo := mid + 1

  ladder := checkableCandidate(f, lo)
  if lo = 0:
    minimumReceipt := {zeros: 0, previousFailure: none}
  else:
    failure := exactFailureOfCandidate(f, lo - 1)
    minimumReceipt := {zeros: lo, previousFailure: failure}
  return ladder, minimumReceipt

```

4.4 A checkable minimality receipt

A success certificate need not be trusted to be short. Minimality is a separate claim, checked through a separate receipt. The replay path first checks the ladder’s mathematical validity. It then verifies that its multiset is precisely the mandatory nonzero factors plus the reported number of zeros. If that number is positive, it reconstructs the preceding candidate and checks one of two exact failure kinds: a negative anchor, or a negative coefficient in its ordinary-polynomial terminal. Monotonicity and the forced-factor lemma give the lower bound.

The minimum receipt never changes whether the theorem is true. Rejecting an incorrect minimum receipt need not discard an otherwise valid proof certificate; the two claims should remain separate in Forge’s interface. The experiment runner requires both because it is testing the optimizer as well as the prover.

4.5 Complexity and the correct sense of optimality

The dense coefficient count is $N = \sum m_\lambda$. A cofactor step visits at most N coefficients. A full candidate has at most N steps and costs $O(N^2)$ rational arithmetic operations. Canonical feasibility costs $O(N^2)$, and binary search uses $O(\log(m_0 + 2))$ candidate checks. Hence a straightforward implementation uses

$$O(N^2 \log(m_0 + 2))$$

rational operations, rather than exploring (7). One candidate can be evaluated with $O(N)$ working coefficient storage, in addition to the input and output. Numerators and denominators may grow: these are arithmetic-operation bounds, not unit-cost bit-complexity bounds.

“Shortest” means the number of T_c steps under Definition 4.1. It does not mean minimum coefficient bit length, shortest rendered proof term, minimum Lean checking time, or shortest proof in a richer calculus. A terminal polynomial proved positive by Forge’s SOS worker might allow a different shorter certificate. That richer checker remains sound, but the minimum theorem above would no longer describe it. An implementation should attach the grammar identifier to every optimality claim.

The initial bounded breadth-first routine is retained as a reference implementation. When its state cap is reached, it reports budget exhaustion if it finds no proof; a proof found after resource pruning need not be globally shortest. The experiments use a cap above the complete tested grids, and the production minimality claim comes from the new theorem and receipt, not from a bounded-search assumption.

5 Worked certificates and what they buy

5.1 Arbitrarily high Taylor truncations need one step

For $n \in \mathbb{N}$ and rational $r > 0$, let

$$f_n(x) = e^{rx} - \sum_{j=0}^n \frac{(rx)^j}{j!}.$$

Direct differentiation gives

$$T_r f_n = \frac{r^{n+1}}{n!} x^n \in \mathcal{C}_+, \quad f_n(0) = 0. \quad (9)$$

Thus every concrete truncation has a one-step ladder, independent of n . Repeated ordinary differentiation would take many more steps before the exponential and the polynomial comparison become easy. The compiler finds the better operation because the nonzero mode forces one cofactor r and no zero cofactors are needed.

There is a parallel alternating family:

$$h_n(x) = (-1)^{n+1} \left(e^{-rx} - \sum_{j=0}^n \frac{(-rx)^j}{j!} \right).$$

Its cofactor is $-r$, and

$$T_{-r}h_n = \frac{r^{n+1}}{n!}x^n \in \mathcal{C}_+, \quad h_n(0) = 0.$$

Both families are implemented for $n = 0, \dots, 15$ and four rational rates. These 128 instances test exact factorial coefficients, negative rates, parity signs, and high-order cancellation. They are not 128 unrelated theorem families.

The mathematics above proves the family uniformly in n . The prototype reifies *concrete* polynomial coefficient arrays; it does not synthesize a Lean proof with a symbolic natural-number truncation parameter. Such a parametric theorem can be added once and instantiated, or handled by Forge's existing induction machinery. Confusing concrete instance generation with parametric induction would overstate the implementation.

5.2 An exponential majorant for binomial powers

Let $n \geq 1$, $r > 0$, and $u(x) = 1 + rx/n$. For

$$f(x) = e^{rx} - u(x)^n$$

one obtains

$$T_r f = ru^n - ru^{n-1} = \frac{r^2}{n} x u^{n-1} \in \mathcal{C}_+.$$

The binomial expansion of u^{n-1} has nonnegative rational coefficients, and $f(0) = 0$. Hence

$$\left(1 + \frac{rx}{n}\right)^n \leq e^{rx} \quad (x \geq 0).$$

The corpus includes $n = 1, \dots, 12$ at the same four rates. The search is not given this derivation as a special theorem; it computes the cofactor residual from the expanded input.

5.3 A fully expanded four-step certificate

Consider

$$f(x) = e^{2x} - (2 + x^2)e^x + 1. \tag{10}$$

The emitted cofactors are $(1, 1, 1, 2)$. Every intermediate function can be displayed:

$$\begin{aligned} g_0 &= e^{2x} - (2 + x^2)e^x + 1, \\ g_1 &= e^{2x} - 2xe^x - 1, \\ g_2 &= e^{2x} - 2e^x + 1, \\ g_3 &= e^{2x} - 1, \\ g_4 &= 2. \end{aligned}$$

The first three transitions are $g_{i+1} = g'_i - g_i$ and the last is $g_4 = g'_3 - 2g_3$. The anchors are

$$(g_0(0), g_1(0), g_2(0), g_3(0), g_4(0)) = (0, 0, 0, 0, 2).$$

Backward application of the scalar rule proves (10) nonnegative. Equivalently,

$$(e^x - 1)^2 \geq x^2 e^x \quad (x \geq 0).$$

There is no floating-point evaluation in this proof. The three occurrences of cofactor 1 are forced by the degree-two coefficient polynomial of e^x , and cofactor 2 is forced by the other nonzero mode. Four steps are therefore minimal in the declared grammar. The full increasing-order pass uses an unnecessary initial zero cofactor and has five steps; the optimal compiler removes it.

This example is also a useful source-normalization test. A Lean user might state it as $(\text{Real.exp } x - 1)^2 \geq x^2 \text{Real.exp } x$, while the worker stores (10). The front end must prove that these expressions agree, using the exponential addition law and ring normalization, rather than compare printed strings.

5.4 Hyperbolic inequalities with exact contact at zero

Put $S = \sinh x$ and $C = \cosh x$. The following residuals reduce to exponential polynomials and produce the listed certificates. A superscript on a cofactor in the table denotes its multiplicity, not exponentiation of the differential operator's argument.

Residual	Sorted cofactors	Nonzero anchors; terminal
$x(C + 2) - 3S$	$(-1, -1, 1, 1)$	All anchors zero; terminal $2x$
$S^2C + xS - 2x^2C$	$(-3, (-1)^3, 1^3, 3)$	$g_6(0) = 128, g_7(0) = 384$; terminal 0
$S^3 - x^3C$	$(-3, (-1)^4, 1^4, 3)$	$g_7(0) = 336, g_8(0) = 1344, g_9(0) = 3072$; terminal 0

Table 3: Exact unscaled certificates. All omitted anchors are zero.

For $x > 0$, division by the proved positive quantities x , x^2C , and x^3 gives, respectively,

$$\begin{aligned} \frac{\sinh x}{x} &\leq \frac{\cosh x + 2}{3}, \\ \left(\frac{\sinh x}{x}\right)^2 + \frac{\tanh x}{x} &\geq 2, \\ \left(\frac{\sinh x}{x}\right)^3 &\geq \cosh x. \end{aligned}$$

The normalization identity for the second inequality uses $\tanh x = S/C$ and $C > 0$. These denominator side conditions belong in the Lean source bridge. They are not inferred from an unguarded cancellation in the prototype.

The last two residuals have their first nonzero Taylor terms in degrees six and seven. The ladder accounts for all preceding zeros exactly. The opposite cofactor ordering fails for these examples even though its full annihilator is the same. This is the concrete reason for the exchange theorem, and supplies eight failures of the decreasing-order ablation after the four argument scalings are included.

5.5 A rational envelope without floating point

For

$$p(x) = (x - 2)e^x + x + 2,$$

the compiler emits two cofactors $(1, 1)$:

$$T_1 p = e^x - x - 1, \quad T_1^2 p = x.$$

All anchors are zero. Hence $p(x) \geq 0$ on the right ray. On the additional domain $0 \leq x < 2$, this gives

$$e^x \leq \frac{x+2}{2-x}.$$

The half-line certificate and the bounded-domain rational envelope are different statements. Only the former is directly represented by the current prototype; the latter follows by a host-side positivity proof for the denominator. A normalizer that dropped $x < 2$ would reverse an inequality in the wrong region.

5.6 An explicit success language, not a hidden universal claim

The controls

$$f_{q,\varepsilon}(x) = (e^x - q)^2 + \varepsilon, \quad q \in \{3/2, 2, 3, 4\}, \quad \varepsilon \in \{0, 1/10000\} \quad (11)$$

are nonnegative. Nevertheless their first canonical cofactor is zero, because of the constant mode, and

$$f'_{q,\varepsilon}(0) = 2 - 2q < 0.$$

The canonical obstruction is therefore immediate. The $\varepsilon = 0$ instances have an interior zero; the positive- ε instances show that even strict positivity does not imply membership in the finite grammar. These are successful *negative language decisions*, not failed attempts to disprove a true theorem.

For Forge, the useful policy is to leave the original goal alive and offer the factorized expression to the existing polynomial worker. That worker only needs the algebraic fact that a square is nonnegative; it need not know any calculus. This is genuine cooperation between different certificate languages, rather than a reason to inflate the new worker's scope.

6 Coupled differential certificates

Evidence status: The constant-rational-matrix fragment is implemented and tested for supplied residual vectors. The general theorem is proved below; its Lean formalization and automatic residual discovery are not implemented.

6.1 Why a vector rule is useful

Some useful inequalities naturally prove one another's derivatives. For example, with

$$H_0(x) = \sinh x - x, \quad H_1(x) = \cosh x - 1,$$

one has

$$H'_0 = H_1, \quad H'_1 = H_0 + x, \quad H_0(0) = H_1(0) = 0.$$

Trying to prove $H_0 \geq 0$ by assuming $H_1 \geq 0$, and then proving $H_1 \geq 0$ by assuming $H_0 \geq 0$, is circular. The correct object is a single vector theorem whose hypotheses license both dependencies simultaneously.

A real matrix is *Metzler* if its off-diagonal entries are nonnegative. Its diagonal entries may have either sign. The distinction matters: at a boundary where component i is zero, the term $A_{ii}H_i$ vanishes, so the sign of the diagonal is irrelevant to the inward-pointing condition.

Theorem 6.1 (Positive-system certificate). *Let $H : [0, \infty) \rightarrow \mathbb{R}^d$ be continuously differentiable, let $A : [0, \infty) \rightarrow \mathbb{R}^{d \times d}$ and $b : [0, \infty) \rightarrow \mathbb{R}^d$ be continuous, and suppose*

$$H'(x) = A(x)H(x) + b(x). \quad (12)$$

Assume $H_i(0) \geq 0$, $b_i(x) \geq 0$, and $A_{ij}(x) \geq 0$ for $i \neq j$, on the whole right ray. Then $H_i(x) \geq 0$ for every component and every $x \geq 0$.

Proof. Fix $T > 0$. Continuity and compactness give a real number K strictly larger than every row sum $\sum_j A_{ij}(x)$ for $x \in [0, T]$. For $\varepsilon > 0$, define

$$w_i(x) = H_i(x) + \varepsilon e^{Kx}.$$

Then $w_i(0) > 0$, and

$$w'_i = (Aw)_i + b_i + \varepsilon e^{Kx} \left(K - \sum_j A_{ij}(x) \right).$$

If some component first reached zero at $t \in (0, T]$, all components of $w(t)$ would be nonnegative. At a zero component i , its diagonal contribution vanishes; the off-diagonal contributions and b_i are nonnegative, and the last term is strictly positive. Thus $w'_i(t) > 0$. But a differentiable function positive immediately to the left of its first zero has derivative at that zero at most zero, a contradiction. Therefore all w_i stay positive on $[0, T]$. Letting ε decrease to zero gives $H_i \geq 0$ there. Since T was arbitrary, the conclusion holds on the right ray. $\square$

The proof does not assume the conclusion during any individual row check. The matrix sign conditions and a single general theorem are what make the feedback sound. In a Lean implementation, the scalar ladder theorem is a particularly simple triangular instance; a separate vector theorem is needed for genuine cycles.

6.2 The implemented certificate and row algorithm

The prototype restricts A to a constant rational matrix and requires each forcing component b_i to belong to $\mathcal{C}_+$. The problem provides the residual functions $H_i \in \mathcal{E}$. A certificate contains A , the forcing polynomials, and the rational anchors. Replay verifies

$$H'_i - \sum_j A_{ij} H_j = b_i$$

by exact sparse exponential-polynomial equality, checks $A_{ij} \geq 0$ off the diagonal, checks coefficient positivity of every b_i , and recomputes every anchor.

The search chooses a row of A at a time. For fixed residuals, each row is independent of the others as an algebraic synthesis problem. For each tuple of candidate coefficients, it forms

$$b_i = H'_i - \sum_j a_j H_j$$

and accepts the first tuple whose residual is in $\mathcal{C}_+$. A finite row-attempt budget bounds this enumeration. The tested dictionaries at scale r are

$$a_i \in \{-2r, -r, 0, r, 2r\}, \quad a_j \in \{0, r, 2r\} \quad (j \neq i).$$

At most 10,000 row attempts are allowed. Exhausting that budget or the dictionary yields unknown for this synthesis request, not a proof that no positive-system certificate exists.

```

searchPositiveSystem(H, diagonalDictionary, offDiagonalDictionary):
  reject this search path if any exact anchor H[i](0) is negative
  for each row i:
    for each bounded coefficient tuple a:
      b := derivative(H[i]) - sum_j a[j] * H[j]
      if b has only rate zero and nonnegative coefficients:
        retain row a and forcing b
        continue with next row
    if no row retained: return Unknown
  return the matrix, forcing, and anchor certificate

```

This is concrete but intentionally modest. It does not search for the residual vector itself. A useful production extension is to form a small residual bank from differences in the original goal, already proved approximants, and cofactor derivatives. That proposal should be evaluated separately; it is not part of the recorded 40-case success count.

6.3 A parametric family behind the 40 tests

For $r > 0$ and $n \geq 0$, define

$$\begin{aligned}
 H_S(x) &= \sinh(rx) - \sum_{j=0}^n \frac{(rx)^{2j+1}}{(2j+1)!}, \\
 H_C(x) &= \cosh(rx) - \sum_{j=0}^n \frac{(rx)^{2j}}{(2j)!}.
 \end{aligned}$$

Then

$$\begin{pmatrix} H_S \\ H_C \end{pmatrix}' = \begin{pmatrix} 0 & r \\ r & 0 \end{pmatrix} \begin{pmatrix} H_S \\ H_C \end{pmatrix} + \begin{pmatrix} 0 \\ r(rx)^{2n+1}/(2n+1)! \end{pmatrix}, \quad H_S(0) = H_C(0) = 0.$$

The coefficient matrix is Metzler and the forcing is coefficient-positive. The tested corpus instantiates this at four rational rates and $n = 0, \dots, 9$. The enumerator finds the rows from the residuals rather than receiving the matrix as its answer.

These examples can also be tackled by scalar ladders. Their role is to validate an additional certificate representation that can export several inequalities together and express feedback without spurious circular proof edges. The results do not establish that the vector worker solves a larger tested set than the scalar worker.

6.4 A scalable replacement for coefficient enumeration

For fixed H_i , synthesis of a constant matrix row is a rational linear feasibility problem. Coefficients of all nonzero exponential modes in $H_i' - \sum_j a_j H_j$ must vanish. Coefficients of its zero mode must be nonnegative, as must the off-diagonal a_j . The diagonal variable is unrestricted. These are linear equations and inequalities in the row entries.

A production search can therefore use an exact rational feasibility solver, or a numerical solver only to propose a support followed by an exact feasibility check. SymPy's rational polynomial domains are a concrete development option for coefficient extraction and exact linear algebra [16]. No such solver is required for the delivered package: row enumeration and all checking run with the Python standard library. The certificate format does not change when the proposer is replaced.

For variable matrices $A(x)$ with polynomial entries, a fixed degree bound produces a similar coefficient-matching problem, but off-diagonal positivity then needs a certificate rather than a scalar sign. Forge’s existing polynomial sign workers can supply those premises. This is an explicitly designed extension of the vector theorem, not an executed result. Nonpolynomial entries, domain guards, and nonzero anchors must likewise have their own proof obligations.

7 Exact analytic enclosures and point counterexamples

Evidence status: Implemented using rational arithmetic. The enclosure proposer and checker share the same numerical kernel; a separate Decimal computation supplies finite sanity checks, not a proof of that kernel.

7.1 A small exponential enclosure with a proved tail

A ladder obstruction says nothing about the sign of the original function. To produce a genuine counterexample, the worker must exhibit a point in the original domain and prove that the original expression is negative there. The prototype does this with rational enclosures of exponentials.

For $0 \leq u \leq 1/2$ and $n \geq 0$, define

$$S_n(u) = \sum_{j=0}^n \frac{u^j}{j!}, \quad a_{n+1} = \frac{u^{n+1}}{(n+1)!}, \quad \rho = \frac{u}{n+2}.$$

Lemma 7.1 (Geometric tail enclosure). *For the above parameters,*

$$S_n(u) \leq e^u \leq S_n(u) + \frac{a_{n+1}}{1-\rho}. \quad (13)$$

Proof. The exponential series has nonnegative terms for $u \geq 0$, so its partial sum is a lower bound. For $j \geq n+1$, consecutive omitted terms have ratio $u/(j+1) \leq \rho < 1$. Thus the tail is at most the geometric series $a_{n+1} \sum_{k \geq 0} \rho^k = a_{n+1}/(1-\rho)$. $\square$

For arbitrary rational t , choose $k \geq 0$ such that $u = |t|/2^k \leq 1/2$. Compute (13), square the positive interval k times, and, if $t < 0$, take the reciprocal with reversed endpoints. This uses only rational arithmetic and the elementary identities

$$e^{2v} = (e^v)^2, \quad e^{-v} = 1/e^v.$$

The prototype allows degrees $0 \leq n \leq 64$, $|t| \leq 64$, and an input-denominator bit-length cap of 128. Unsupported inputs fail the numerical worker; these limits are not assumptions in the mathematical enclosure lemma.

There are two important advantages over printing a decimal approximation. Every endpoint is an exact rational number, and the tail bound has a short mathematical justification. There is also a cost: exact squaring can create large numerators and denominators. A future implementation may use proved outward-rounded dyadics for compactness, but that would be a different numerical kernel requiring its own correctness proof.

7.2 Interval evaluation of the original normal form

Given a rational interval $[a, b]$, each monomial term $cx^j e^{\lambda x}$ is enclosed by interval multiplication of an enclosure for x^j and one for $e^{\lambda x}$. The exponential is monotone, so the two rate-scaled endpoints suffice. For even j and an interval crossing zero, the lower bound on x^j is zero; otherwise the appropriate endpoint powers are used. Multiplication considers all four endpoint products. A reciprocal is permitted only when the input interval excludes zero.

Summing term enclosures yields a valid but potentially wide enclosure of (1). This is a natural interval extension, not a Taylor model of the full expression. It does not preserve correlations between repeated occurrences of x , and it can be inconclusive even when the target is far from changing sign. Subdivision or a better normal form may help; changing a failed lower bound into zero would not.

The same limitations explain why a polynomial square worker is still valuable. Expanding $(e^x - 2)^2$ and applying interval arithmetic loses the obvious square structure. The existing symbolic worker can establish nonnegativity without evaluating an exponential at all.

7.3 A point certificate has a different logical meaning

The point search tries rational candidates $0, 1/4, 2/4, \dots, 8$, and increases the Taylor degree through 8, 16, 32. These are proposal heuristics. A point certificate contains the selected rational point, the degree, and the resulting rational enclosure. Its checker:

1. decodes the original externally supplied positivity problem;
2. checks that the point lies on its right ray;
3. recomputes the enclosure from the original expression; and
4. accepts only if the upper endpoint is strictly negative.

An accepted point certificate mathematically refutes the universal nonnegativity statement. It is not merely failure of a proof language. Conversely, failure to find such a point within the list and degree budgets says only unknown. In particular, the eight true controls in (11) return a ladder obstruction and no point counterexample. The two outcomes must never be conflated.

In an eventual Lean interface, a point counterexample could justify a theorem of the form $\neg(\forall x \geq 0, 0 \leq f(x))$, once the enclosure evaluator and its source bridge are verified. A tactic trying to prove $f(x) \geq 0$ cannot use that negative theorem to close the positive goal. It can report a diagnosis or discharge a separately requested negation.

8 Root certificates as analytic witness plans

8.1 The target includes the interval

The root lane receives an expression $f \in \mathcal{E}$ and an *externally specified* rational interval $[a, b]$, with $a < b$. Its intended conclusion is

$$\exists! x \in [a, b], \quad f(x) = 0, \tag{14}$$

where uniqueness is restricted to that interval. The notation means that there is exactly one real number satisfying both interval membership and the equation.

The producer may not replace a requested global uniqueness theorem by a convenient local interval and claim the original goal has been solved. For a goal asking merely for a root in a larger domain, a smaller bracket is useful only after the host proves that the bracket is contained in the requested domain. This source-to-goal condition is separate from numerical validity.

8.2 Endpoint signs plus an exact derivative cover

A certificate chooses a direction $d \in \{-1, 1\}$ and proves

$$df(a) < 0 < df(b), \quad df'(x) > 0 \quad (x \in [a, b]). \quad (15)$$

Strict endpoint signs are certified by rational enclosures. The derivative is recomputed from the original normal form, then enclosed on a finite subdivision tree. Each leaf carries a Taylor degree and a rational interval with strictly positive lower endpoint after multiplication by d .

Coverage is checked structurally. The root node inherits exactly $[a, b]$. A split at rational m must satisfy $a < m < b$, and its children inherit $[a, m]$ and $[m, b]$. Leaves do not get to choose their own smaller domains. The checker limits the total tree to 2,048 visited nodes; the proposer uses midpoint splits with maximum depth eight. These checks prevent omitted gaps and detached successful subintervals from being mistaken for a full derivative proof.

Theorem 8.1 (Unique-root certificate). *For a continuous function on $[a, b]$, differentiable there, conditions (15) imply (14), with the root in fact lying in (a, b) .*

Proof. Apply the intermediate value theorem to df , whose endpoint values have opposite signs. The root cannot equal either endpoint because their signs are strict. The derivative condition and the mean value theorem make df strictly increasing on the interval. Two distinct roots would contradict strict monotonicity. $\square$

This theorem makes an interval certificate into a useful existence proof. It does not need an exact symbolic closed form for the root. The Mathlib intermediate-value and derivative-monotonicity APIs provide concrete formalization targets [11, 10].

8.3 Concrete tested root problems

The corpus includes eight equations

$$xe^x = q, \quad q \in \{1/8, 1/4, 1/2, 1, 2, 3, 4, 8\},$$

with distinct supplied rational brackets, four equations $e^x = q$ with $q = 2, 3, 5, 10$, and four additional brackets. Two of the latter isolate two distinct positive roots of $e^x = 3x$ separately:

$$[1/2, 1], \quad [3/2, 2].$$

The derivative certificate is decreasing on the first bracket and increasing on the second. This pair is a deliberate check against promoting interval uniqueness to global uniqueness.

The remaining two examples are $e^x = 2x + 1$ on $[1, 3/2]$ and $e^{-x} = x$ on $[1/2, 3/5]$. For $xe^x = 1$, the recorded bracket is $[1/2, 3/5]$. The implementation never requires a Lambert- W library: the defining equation, continuity, endpoint enclosures, and derivative sign suffice.

8.4 An exact existence proof is not a computable real value

The root certificate proves that an exact real number exists in a bracket. It does not identify the midpoint, a decimal expansion, or either endpoint as that root. A Lean proof of an existential proposition can be constructed by the intermediate value theorem. A computational function returning arbitrary-precision approximations would need an additional representation of computable reals, nested brackets, a convergence argument, and an error specification. None of those is delivered by a single bracket.

For a synthesis system, there are therefore two distinct interfaces. A proposition-valued hole such as

$$\exists x \in [1/2, 3/5], xe^x = 1$$

can be filled by an existence theorem. A data-valued request for a machine number whose exact real interpretation solves the equation cannot be filled by pretending that the bracket midpoint is exact. A noncomputable choice of the unique real is possible in a sufficiently classical Lean context, but its computational and axiom implications must be explicit. Flow does not change the user’s permitted axiom policy.

8.5 Deliberate incomplete cases

The current root worker requires strict endpoint signs and a strict derivative-sign cover. It returns unknown for an endpoint root, an even-multiplicity interior root with no sign change, or a simple root whose derivative cannot be bounded away from zero by the chosen interval extension. A function may have a unique root even when these sufficient conditions fail. Increasing numerical budgets is sometimes useful, but it is not a completeness theorem.

The guard tests include an endpoint-root case, a forged split, an altered external bracket, and a reversed monotonicity direction. These test different failure modes. They do not amount to a proof that the checker is hardened against every malformed or adversarial byte sequence.

9 A concrete Lean implementation plan

9.1 Keep discovery outside the trusted mathematics

Flow should be a new worker family below Forge’s existing obligation controller, not a replacement for it. It recognizes a selected analytic fragment, produces a certificate, and exports a theorem about the original Lean expression after reconstruction. The current `grind` documentation remains the baseline for the local saturation engine [8]; this proposal does not depend on changing its internals.

The shortest-ladder algorithm can remain an untrusted program. To accept a positive theorem, Lean needs the soundness of the certificate checker and the source-to-normal-form bridge, not a proof that the compiler found the shortest certificate or searched completely. Canonical obstruction and minimality are separate diagnostic claims. Formalizing their theorems is valuable, but it should not delay the first useful positive-proof path.

9.2 Suggested module boundaries

The following are proposed module names, not modules claimed to exist in the current repository.

Module	Responsibilities and exact acceptance target
<code>Forge/Flow/ExpPoly.lean</code>	Rational mode representation; denotation in $\mathbb{R}$; normalization; exact addition, multiplication, derivative, and anchor. Prove that each operation preserves its denotation.
<code>Forge/Flow/Cofactor.lean</code>	Prove the scalar integrating-factor rule and coefficient-positive polynomial evaluation. No search logic.
<code>Forge/Flow/Ladder.lean</code>	Executable checker and theorem: a true check implies nonnegativity of the denotation on the right ray.

Module	Responsibilities and exact acceptance target
Forge/Flow/Reify.lean	Construct normal-form data together with equality to the selected source expression. Preserve the selected real argument and all local hypotheses.
Forge/Flow/Compile.lean	Optional native implementation of increasing-order feasibility and binary-search shortening. Positive acceptance still goes through Ladder .
Forge/Flow/Optimality.lean	Optional formalization of exchange, forced multiplicities, zero-count monotonicity, and grammar-relative minimum receipts.
Forge/Flow/PositiveSystem.lean	A single vector positivity theorem plus checking of constant rational matrices, forcing, and anchors.
Forge/Flow/ExpBounds.lean	Rational interval operations, geometric exponential tail, scaling, squaring, reciprocal, and denotation enclosure theorem.
Forge/Flow/Roots.lean	Checked inherited-interval coverage; endpoint and derivative signs; application of IVT and strict monotonicity.
Forge/Flow/Adapter.lean	Convert only supported goals; discharge domain and normalization obligations; return ordinary Lean proof terms to Forge.

Table 4: A small mathematical kernel can be implemented before any sophisticated analytic search.

Use a simple specification representation first, such as a list of rational rates and rational polynomials. For example, denotation can be defined by a finite sum of P_λ evaluated in $\mathbb{R}$ and multiplied by $\text{Real.exp}((\lambda : \mathbb{R})x)$. Arrays, maps, and optimized rational arithmetic can be refinements of this model. An optimized container should not be mixed into the first proof of the analytic theorem unless it demonstrably simplifies the development.

9.3 The first four formal theorems

The following signatures describe the obligations semantically; the names beginning with **Flow**. are proposed API names, not verified declarations.

```

Flow.derivative_sound(p, x):
  HasDerivAt (denote p) (denote (derive p) x) x

Flow.anchor_sound(p):
  denote p 0 = castToReal (anchor p)

Flow.terminal_sound(p):
  coefficientPositivePolynomial p = true ->
  forall x, 0 <= x -> 0 <= denote p x

Flow.ladder_sound(p, certificate):
  checkLadder p certificate = true ->
  forall x, 0 <= x -> 0 <= denote p x

```

The derivative proof is by finite-sum and polynomial induction, using the product rule and the real exponential derivative. The cofactor identity is a rational-algebra consequence. The

terminal theorem is induction over nonnegative monomials. The ladder theorem is induction over the certificate chain, using the scalar rule backwards.

A direct proof-producing implementation is also viable before reflection: construct each derivative identity as an ordinary Lean proof, normalize its polynomial coefficients, establish the rational seed, then apply the scalar theorem. This avoids trusting a new Boolean checker while the front end is being stabilized. Reflection becomes attractive when per-step proof-term overhead is measured, rather than assumed to dominate.

9.4 Concrete Mathlib targets inspected

The current generated API documentation was inspected for the following ingredients [9, 10, 11]. These are library targets, not evidence that the proposed modules compile against Forge’s pin.

Import family	Relevant declarations or role
Mathlib.Analysis.SpecialFunctions.ExpDeriv	Real.hasDerivAt_exp, HasDerivAt.exp, Real.deriv_exp, and smoothness of the exponential.
Mathlib.Analysis.Calculus.Deriv.MeanValue	monotoneOn_of_deriv_nonneg, monotoneOn_of_hasDerivWithInAt_nonneg, and strictMonoOn_of_deriv_pos.
Mathlib.Topology.Order.IntermediateValue	intermediate_value_Icc and its reversed-endpoint variant.
Rational and polynomial infrastructure	Exact casts, polynomial evaluation, finite sums, rational order, and ring normalization; prove the denotation bridge before optimizing representation.

Table 5: Analytic proof infrastructure to reuse rather than reimplement.

For the scalar rule, apply the monotonicity theorem to $x \mapsto e^{-cx}g(x)$ on the convex set $[0, \infty)$. Supply continuity on the set and differentiability on its interior, deriving the latter from explicit `HasDerivAt` proofs. The product derivative becomes $e^{-cx}(g' - cg)$ after algebraic normalization. This avoids introducing an explicit improper integral or an ODE existence theorem.

For a root bracket, obtain zero in the image of $[a, b]$ through `intermediate_value_Icc`, using the endpoint sign bounds. Derivative-sign coverage gives strict monotonicity; injectivity on the interval supplies uniqueness. The exponential-tail theorem requires formalizing Lemma 7.1 from the exponential series and a geometric-series bound. No existing Mathlib theorem with exactly that checker signature is claimed here.

Forge’s README pins Lean v4.34.0 and Mathlib revision 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa [1]. The first implementation task is to compile these ingredients against those versions and record the actual dependencies. Current documentation and a repository pin are not interchangeable evidence of API compatibility.

9.5 The source bridge is part of the proof

Suppose the current local goal is $0 \leq F(u)$, and the worker proves $0 \leq \text{denote}(p)(t)$ whenever $t \geq 0$. A correct adapter needs all of

$$0 \leq u, \quad F(u) = \text{denote}(p)(u), \quad \forall t \geq 0, 0 \leq \text{denote}(p)(t). \quad (16)$$

The final proof is then equality transport and specialization. A mathematically valid certificate for the wrong p is useless. The prototype tests altered original problems, but it does not build the Lean equalities in (16).

The bridge should normalize products of exponentials using proved addition laws and reduce hyperbolic functions using their proved definitions. It should carry division-sign obligations

explicitly. For example, converting the second inequality after Table 3 to its residual requires $x > 0$ and $\cosh x > 0$. The worker’s proof of the expanded residual does not establish those guards. A failed guard means the original goal remains open.

The simplest front end should select one real expression as its scalar argument rather than treating every free variable as an independent coordinate. Other parameters must be concrete rational data, or be specialized through a separately proved family theorem. A symbolic parameter $r > 0$ does not fit a representation whose rate field is `Rat`; pretending otherwise would turn a concrete instance checker into an unsupported parametric solver.

9.6 Routing and result types specific to this lane

The following result variants prevent a particularly dangerous analytic confusion:

```
FlowResult =
  PositiveLadder(certificate, optionalMinimumReceipt)
| PositiveSystem(certificate)
| NegativePoint(originalProblem, certificate)
| UniqueRoot(originalIntervalProblem, certificate)
| NoLadderInGrammar(grammarId, obstruction)
| Unknown(reason)
```

A positive ladder can contribute a theorem about the original expression. A grammar obstruction can only stop this compiler branch or inform routing. A negative point can support a refutation after verified enclosure replay. A unique root supplies an existential theorem restricted to the externally bound interval. These permissions are local mathematical consequences of the certificate families, not a redesign of Forge’s already specified trust architecture.

A reasonable dispatch policy is: try existing cheap algebra first; recognize the exponential-polynomial fragment; run the optimal ladder compiler; use a supplied small residual bank for the positive-system worker; and use bounded enclosures for strict local goals, diagnostics, or root requests. If a ladder fails at a negative anchor, route to algebraic factorization or the existing polynomial sign engine before attempting expensive numeric subdivision. No new global scheduler is required.

9.7 How Leant and Djex would use the new worker

Leant’s pinned README describes a Djex-backed synthesis REPL that re-elaborates candidates in Lean. Djex combines an LJIT-based propositional engine with a ranked heuristic expression synthesizer and scoped support for richer types. Both projects explicitly distinguish accepted fragments from bounded inconclusive searches [6, 7]. Forge has already incorporated many of their verification-boundary lessons [5]; those lessons are not presented as new here.

The analytic addition is a concrete provider of *specification evidence*. Consider a synthesized candidate expression whose required property is

$$\forall x \geq 0, \quad q(x) \leq e^x.$$

If q is a rational polynomial, the remaining property is directly a Flow problem. The term synthesizer proposes the candidate; Flow produces a theorem about that exact candidate; Lean connects the theorem to the original specification. A rational point counterexample can reject the candidate without claiming that no other term can satisfy the type.

For existential specifications, the root lane is a provider of an IVT proof plan, not a fabricated data value. A synthesis request that merely needs an inhabitant of an existence proposition can

use it after reconstruction. A request for executable numeric code needs a separate approximation specification. Keeping those interfaces separate is more useful than making the analytic worker appear stronger by conflating them.

Nothing in this proposal closes the neighboring projects' outstanding original integer-indexing or simultaneous two-universe composition queries. Their current ledgers keep those goals open [6, 7]. Flow addresses a different kind of missing capability: analytic correctness obligations that arise after or during term construction.

9.8 Trust and performance gates

The first trustworthy milestone is not a large benchmark. It is one original source goal, one exact certificate, a proved reification equality, and a kernel-checked proof with the intended axiom dependencies. Then repeat with a deliberately altered cofactor, anchor, source expression, and domain; all must fail in the appropriate layer.

After that, measure normalization, certificate construction, reconstruction, and kernel checking separately. A 76% reduction in cofactor count need not yield a 76% reduction in runtime, because normalization or rational coefficient growth may dominate. The compiler should record both its arithmetic dimensions and proof-checking cost once the Lean backend exists. The current Python elapsed times do not answer that question.

10 Executed evidence and its limits

10.1 Environment and record locations

The accepted run used Python 3.13.5 on Linux x86-64. All proof-producing and replay decisions use `Fractions.Fraction`; floating-point numbers appear only in elapsed-time measurements. The rational library represents numerator and denominator exactly [17]. No Lean or Lake executable was available in this environment, and no Lean compilation, axiom audit, or comparison with a Lean tactic was performed.

The full problem and certificate records are in `results/accepted/certificates.json`; the environment, counts, mutation totals, and timings are in `results/accepted/summary.json`. Every record retains its lane and original problem. Re-running the generator requires a new output directory so that it does not silently overwrite the accepted record.

The principal corpus is deterministic and intentionally structured. It is a functionality corpus built for this new fragment, not a representative sample of Mathlib. Its parameterized families were chosen to test exact cancellation, negative rates, nontrivial ordering, terminal compression, and the distinction between theorem truth and language membership.

10.2 Results by logical output

The 196 positive cases comprise 64 positive-exponential Taylor remainders, 64 alternating negative-exponential remainders, 48 binomial majorants, and 20 scaled special examples. The four scales are $1/3$, 1 , $3/2$, 2 . The 40 system cases are two-component hyperbolic truncation systems. The 44 false cases combine twelve exaggerated tangent lower bounds with 32 reversed Taylor inequalities. The eight grammar controls are (11). The sixteen root cases are described in Section 8.

Each positive record was checked through the sparse ladder verifier and the minimum-receipt verifier. The canonical and breadth-first solutions were also obtained on the same inputs, with

Lane and expected outcome	Cases	Correct recorded outcomes
Nonnegative exponential-polynomial ladders	196	196
Coupled positive-system certificates	40	40
Negative rational-point certificates	44	44
True functions outside the ladder grammar	8	8
Unique roots in supplied rational brackets	16	16

Table 6: The fourth row is a checked grammar rejection of true functions, not a theorem refutation. The counts are different kinds of evidence and are not a Lean success rate.

the latter serving as an independent algorithmic cross-check of length. All breadth-first searches here fit inside the configured 50,000-state budget.

10.3 The cofactor-order and compression ablation

Measurement on the same 196 positive problems	Result
Increasing-order feasibility successes	196
Decreasing-order feasibility successes	188
Inputs strictly shortened by the optimal compiler	124
Total steps in increasing-order, early-stopping certificates	1,212
Total steps in minimum certificates	288
Reduction in total cofactor steps	76.24%

Table 7: This is an ablation of Python certificate algorithms, not a comparison with `grind`.

The numerator in the percentage is $1212 - 288 = 924$. The denominator is the total initial cofactor count on the same inputs. It is not the number of theorem statements, bytes, CPU seconds, or Lean kernel reductions. Many Taylor-family improvements have the same algebraic explanation, so the result should not be described as 124 independently discovered proof strategies.

The accepted suite’s recorded elapsed time was approximately 1.15 seconds in this environment, including its searches, checks, and mutations. That is a single aggregate observation on this small corpus. It is not a throughput guarantee, a controlled microbenchmark, or an estimate of an eventual Lean implementation. Per-input timings are retained for inspection rather than used to assert a speedup.

10.4 Targeted invalid certificates

Mutation family	Attempted	Rejected
Increase one ladder anchor by exactly 1	484	484
Increase one system anchor by exactly 1	80	80
Increase a reported negative-point upper bound by 1	44	44
Reverse the root certificate’s direction	16	16
Total targeted invalid mutations	624	624

Table 8: Each mutation is intentionally invalid under the exact receipt format. This is not an exhaustive adversarial-input test.

Increasing an anchor can preserve its nonnegative sign, so rejection demonstrates that the checker enforces equality to the recomputed value, not merely positivity of whatever the producer wrote. Similarly, the point checker recomputes the enclosure rather than trusting its sign. A changed direction cannot reuse unchanged endpoint or derivative receipts.

The unit suite adds 22 test methods covering reduced rational encoding, Boolean and float rejection at rational inputs, duplicate modes, wrong anchors and domains, terminal corruption, external target substitution, false minimality receipts, budget preservation, negative off-diagonal matrix entries, interval edge cases, malformed certificates, and invalid coverage. Those 22 methods include multiple assertions and a 120-case random comparison; the number of test methods is not another count of proved theorems.

10.5 Independent finite checks

The optional `oracle.py` script uses seed 20260915 and generates 200 small exponential polynomials with rates drawn from $\{-1, 0, 1\}$, degree at most one per mode, and coefficients in $\{-3, \dots, 3\}$. It performs the following distinct comparisons:

1. Enumerate every distinct full-annihilator permutation and compare the existence of a valid anchor chain with canonical increasing-order feasibility.
2. Compare the optimal compiler with exhaustive-grid breadth-first search, including the accepted minimum length.
3. Compare dense symbolic derivatives with SymPy 1.14.0, then compare the separate sparse derivative implementation with the same exact normal forms.

All 200 cases passed each comparison. The exhaustive permutation check shares the dense transition arithmetic with discovery, so it is an independent *search strategy*, not an independent arithmetic implementation. The SymPy comparison is the additional arithmetic cross-check. These tests support the implementation; the general conclusions about ordering and optimality rest on the proofs in Section 4.

A further 161 rational arguments $k/8$, $-80 \leq k \leq 80$, were checked against Python Decimal exponentiation at 100-digit working precision. The values lay in the computed rational enclosures. This is numerical sanity evidence only. A finite-precision exponential computation is not the authority for Lemma 7.1, and its successful comparison does not establish a formal error bound for every argument.

The oracle summary is stored in `results/oracle-results.json`. The optional SymPy dependency is not used by any acceptance checker or by the standard-library experiment run.

10.6 Replay without discovery or site packages

The stored certificates replay under

```
cd prototype
python -S replay.py
```

The replay script imports the sparse algebra checker and the enclosure checker, but not the dense model, search module, witness proposer, corpus generator, or SymPy. It reports 196 positive, 40 system, 44 point, 8 obstruction, and 16 root records accepted, with discovery modules absent from the loaded module inventory.

There are still two different separation levels. The scalar and vector algebra is recomputed through a separate sparse representation. The numerical interval proposer and its checker share the same enclosure implementation. The latter replay demonstrates independence from

point and bracket *search*, not independence of the numerical kernel. A formally verified Lean enclosure theorem is the required next step, not an extra label saying “independent” on the current Python function.

10.7 What remains unmeasured

No head-to-head test against `grind`, `nlinarith`, `positivity`, `Aesop`, or external analytic provers was run. Existing tactics may already solve some examples after suitable lemmas or preprocessing; no opposite claim is made. The observed gain is a new executed certificate mechanism relative to the reviewed Forge design, with a proved compiler result and concrete examples.

A future comparison should separate raw source goals from goals supplied with analytic lemmas, use the same exact Lean and Mathlib versions, include false and out-of-grammar controls, record all preprocessing, and time source normalization, search, reconstruction, and checking independently. It should report unique theorem families as well as parameter instances to avoid inflating coverage through repeated scaling.

11 Implementation sequence and acceptance gates

The most useful first delivery to the repository is the scalar proof path, not a large collection of uncompiled interfaces. Its mathematics is short and it already handles the strongest worked examples.

Gate 1: analytic foundation. Compile a proof of the scalar cofactor rule and the derivative and anchor semantics for a simple exponential-polynomial representation. The acceptance test is the four-step certificate for (10), proved about the original Lean expression. A theorem merely about a list of coefficients does not close this gate.

Gate 2: source-bound replay. Implement the restricted reifier, the exact ladder checker, and the equality bridge. Replay all 196 positive records from externally supplied original goals. Mutate source rate, coefficient, scalar argument, domain, and cofactor separately. A passing reflected check without a proved source equality does not close this gate.

Gate 3: coupled residuals. Formalize the constant-matrix positive-system theorem and check the 40 paired residual inputs. A pair of circular assumptions does not close this gate; the vector theorem must be the source of the simultaneous conclusion.

Gate 4: exact enclosures. Formalize rational interval operations and Lemma 7.1; verify all 44 point certificates against their original expressions. Floating-point agreement or a Decimal oracle is not a substitute for this proof.

Gate 5: roots and existential specifications. Reconstruct the 16 root certificates as existence-and-uniqueness theorems on their stated intervals. Include the two separate brackets for $e^x = 3x$ and an endpoint-root rejection. A rational approximation of a root does not close this gate.

Gate 6: diagnostics and optimization. Formalize the exchange and minimum theorems if grammar-negative or optimality claims are to be exposed as checked results. Positive theorem acceptance does not depend on this milestone. After the earlier gates, perform the controlled Lean corpus comparison described above.

These gates are acceptance conditions, not time estimates. They name the exact theorem or integration artifact that is still missing and prevent a nearby easier result from being presented as completion.

12 Reproduction and package guide

12.1 Files

article/forged-flow.tex	main LaTeX source
article/sections/	mathematical and implementation chapters
article/references.tex	source bibliography
article/forged-flow.pdf	compiled article
prototype/forged_flow/	exact model, compiler, checkers, witnesses
prototype/corpus.py	deterministic fixture generators
prototype/run_experiments.py	experiment and mutation runner
prototype/replay.py	search-free replay entry point
prototype/oracle.py	optional independent finite diagnostics
prototype/tests/	unit and regression tests
results/accepted/	original accepted certificate run
results/oracle-results.json	exhaustive and symbolic cross-check counts
results/unit-tests.log	recorded unit-test output
lean/FlowTargets.lean	unproved acceptance-target propositions
lean/README.md	exact scope of the Lean artifact
docs/	provenance, status, history, integration notes
README.md	commands and entry points
reproduce.sh	standard-library reproduction script
build_article.sh	pdfLaTeX build script
LICENSE	terms for the new original material

12.2 Commands

The core package has no third-party dependency. From its root:

```
./reproduce.sh
```

This runs the unit suite, replays the accepted records without site packages, creates a fresh **reproduced-results** directory, generates the corpus there, and replays it. Reusing an already existing output directory is an error, preserving the distinction between accepted and newly generated evidence.

The optional oracle can be run with SymPy available:

```
cd prototype
python oracle.py --out ../oracle-reproduced.json
```

Without SymPy it records that part as not run while retaining the other diagnostics. Do not run the test scripts with Python's assertion-eliminating optimization option. The acceptance checker functions themselves return Booleans; the experiment harness intentionally uses assertions to stop on disagreement.

To compile the article:

```
./build_article.sh
```

The source uses pdfLaTeX and standard TeX packages, without shell escape, external bibliography tooling, or bundled fonts. All URLs in the bibliography identify sources inspected for this proposal; third-party code and papers are not redistributed in the archive.

12.3 Small direct API example

```

from forge_flow.model import ExpPoly
from forge_flow.search import optimal_ladder
from forge_flow.check import check_ladder, check_minimality

#  $\exp(2x) - (2 + x^2)\exp(x) + 1$ 
f = ExpPoly.make({0: [1], 1: [-2, 0, -1], 2: [1]})
result = optimal_ladder(f)
assert result["status"] == "certificate"
assert check_ladder(f.problem(), result["certificate"])
assert check_minimality(f.problem(), result["certificate"],
                        result["minimality"])
print(result["certificate"]["cofactors"])
# [[1, 1], [1, 1], [1, 1], [2, 1]]

```

Each cofactor is a reduced integer pair representing a rational number. The direct API expects integers or `Fraction` coefficients, not floating-point approximations or arbitrary computer-algebra scalar classes. A development display script initially supplied a SymPy scalar and was rejected by this boundary; converting the already exact value to a standard-library rational fixed that script. No mathematical checker condition was weakened.

13 Conclusion

Forge can acquire a substantial analytic capability without becoming a general-purpose transcendental solver. The proposed lane has a narrow, explicit representation; a short analytic soundness theorem; an exact compiler whose feasibility and minimum-length behavior are proved for its certificate grammar; a vector extension for coupled residuals; and separate numerical certificates for counterexamples and root existence.

The strongest result is not simply that a bounded search found several inequalities. It is that the apparent cofactor-order search has a canonical solution, and the remaining shortest-certificate problem reduces to a monotone choice of one integer. That observation turns a potentially exponential exploratory worker into a predictable exact compiler for a useful fragment.

The delivered code establishes that these algorithms execute and their stored certificates replay under the stated tests. The mathematical proofs explain what the certificates mean. A verified Lean source bridge and checker are the remaining steps that would turn this research lane into accepted Forge proofs. Keeping those three levels distinct makes the proposal usable: the next implementer receives algorithms, data, proofs, failure controls, and explicit gates rather than another architecture sketch.

References

- [1] Vladimir Reshetnikov, *Forge*, README, revision c98e47c5f804e92880fc1d0e378c1b95832b685c. Inspected 15 September 2026. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/README.md>.
- [2] Forge, *Implementation status*, same revision. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/docs/STATUS.md>.
- [3] Forge, *Nonlinear arithmetic: exact certificates, not floating-point proofs*, article section 7, same revision. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/07-nonlinear.tex>.
- [4] Forge, *Provenance: what came from where*, article appendix B, same revision. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/B-provenance.tex>.
- [5] Forge, *Ideas from Leant and Djex*, same revision. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/docs/IDEAS-FROM-LEANT-DJEX.md>.
- [6] Vladimir Reshetnikov, *Leant*, README, revision 6bf05ad78c467989e68290f2d08bbbed40802d485. Inspected 15 September 2026. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/README.md>.
- [7] Vladimir Reshetnikov, *Djex*, README, revision e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5. Inspected 15 September 2026. This is the reviewed repository revision, not a claim that it is Leant’s dependency pin. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/README.md>.
- [8] Lean Language Reference, *The grind tactic*. Current documentation inspected 15 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [9] Mathlib contributors, *Mathlib.Analysis.SpecialFunctions.ExpDeriv*, generated API documentation. Inspected 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Analysis/SpecialFunctions/ExpDeriv.html.
- [10] Mathlib contributors, *Mathlib.Analysis.Calculus.Deriv.MeanValue*, generated API documentation. Inspected 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Analysis/Calculus/Deriv/MeanValue.html.
- [11] Mathlib contributors, *Mathlib.Topology.Order.IntermediateValue*, generated API documentation. Inspected 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Topology/Order/IntermediateValue.html.
- [12] André Platzer and Yong Kiam Tan, *Differential Equation Axiomatization: The Impressive Power of Differential Ghosts*, LICS 2018. DOI: 10.1145/3209108.3209147. Author version inspected. <https://symbolaris.com/pub/diffaxiomatic.pdf>.
- [13] Guillaume Melquiond and contributors, *Interval Package for Rocq*, official documentation. Inspected 15 September 2026. <https://coqinterval.gitlabpages.inria.fr/>.
- [14] Lawrence C. Paulson, *The MetiTarski Theorem Prover*, project and publication page. Includes B. Akbarpour and L. C. Paulson, *MetiTarski: An automatic theorem prover for real-valued special functions*, Journal of Automated Reasoning 44(3), 175–205, 2010. <https://www.cl.cam.ac.uk/~lp15/papers/Arith/>.
- [15] J. M. Aldaz, O. Kounchev, and H. Render, *Shape preserving properties of generalized Bernstein operators on Extended Chebyshev spaces*, arXiv:0805.1614, 2008. Abstract inspected for related-work scope; no theorem in this article is attributed to this source without proof. <https://arxiv.org/abs/0805.1614>.
- [16] SymPy contributors, *Introducing the Domains of the poly module*, SymPy 1.14.0 documentation. <https://docs.sympy.org/latest/modules/polys/domainsintro.html>.
- [17] Python Software Foundation, *fractions—Rational numbers*, Python 3.13 documentation. The experiment’s exact interpreter version is recorded separately. <https://docs.python.org/3.13/library/fractions.html>.